PART 2

Large Language Models in Depth

Five chapters moving from how models are built to how they behave: what an LLM is made of, what it can and cannot do, what inference costs, how capability scales, and how reasoning models spend compute at test time.

IN THIS PART

- 9. Anatomy of an LLM
- 10. Capabilities and Hard Limits
- 11. Inference, Latency, Throughput and Cost
- 12. Scaling Laws and Benchmarking
- 13. Reasoning Models and Test-Time Compute

CHAPTER 9

Anatomy of an LLM

In this chapter

Introduction: The Capacity Plan That Was Off by a Factor of 199 . . . 4
9.1 What a Parameter Count Actually Counts . . . 5
9.2 The Anatomy of One Block . . . 6
9.3 A Real Model, Counted Exactly . . . 7
9.4 Width, Depth and the Two Directions of Scale . . . 8
9.5 Mixture of Experts: Total Parameters and Active Parameters . . . 9
9.6 Precision: What a Number Weighs . . . 10
9.7 The Served Model: Seven Components, Three You Control . . . 12
9.8 The KV Cache Is the Other Model . . . 13
9.9 What Model Size Does and Does Not Predict . . . 14
9.10 Model, Checkpoint, Deployment, Alias . . . 15
9.11 What Actually Changes Between Generations . . . 17
9.12 Where the Anatomy Sits in Your Architecture . . . 18
9.13 Code Walkthrough: Counting a Model From Its Specification . . . 19
9.14 Mini Project: The Anatomy Calculator . . . 21
9.15 Hands-on Labs . . . 24
9.16 Lab Solutions . . . 25
9.17 Best Practices . . . 26
9.18 Common Mistakes . . . 27
9.19 Performance Considerations . . . 29
9.20 Security Considerations . . . 30
9.21 Real Production Examples . . . 31
9.22 Certification Notes . . . 31
9.23 Practice Questions . . . 32
9.24 Answers and Explanations . . . 34
9.25 Ten Practical Exercises . . . 35
9.26 Five Challenge Labs . . . 36
9.27 Interview Questions . . . 36
9.28 Summary . . . 36
9.29 Glossary . . . 38
9.30 Further Reading . . . 39

Part 2 of 13 · Chapter 9 of 64. Next: Chapter 10 — Capabilities and Hard Limits.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. State exactly **what a parameter count counts**, and derive a model's total from its published dimensions rather than accepting the marketing number.
2. Attribute a model's parameters to the **embedding, attention and feed-forward matrices** of Chapter 4's section 4.8, and explain why the feed-forward layers dominate.
3. Distinguish **width scaling from depth scaling**, and explain why one is quadratic and the other linear.
4. Explain **mixture-of-experts** models and why **active parameters**, not total parameters, govern latency and compute.
5. Compare **fp32, bf16, fp8 and int4** precision, compute the weight memory each implies, and say what quantisation costs.
6. Name the **seven components of a served model** and say which of them you, as an architect, can influence.
7. Size a **KV cache** against context length under MHA, GQA and MQA, and derive the resulting concurrency ceiling on a given accelerator budget.
8. Distinguish a **model, a checkpoint, a deployment and an API alias**, and explain why dateless pinned model IDs change how you manage upgrades.

NOTE

Prerequisites

Chapter 3 for parameter counting in a fully-connected layer (section 3.3) and for the distinction between capacity and capability. **Chapter 4** above all — the decoder-only stack of section 4.1, the parameter accounting of section 4.8, and the KV cache, prefill and decode of section 4.9. **Chapter 5** for the MHA / GQA / MQA arithmetic in section 5.7 and Figure 5.3, which this chapter turns into a calculator. **Chapter 6** for tokenizers as fixed artefacts, because the tokenizer is part of the deployment. **Chapter 7** for the training stages that produce a checkpoint in the first place.

Chapter 4 gave you the mechanism. Chapter 5 gave you the memory arithmetic. This chapter puts a specification in one end and gigabytes, dollars and a concurrency number out of the other. It opens Part 2, where we stop asking *how does a transformer work* and start asking *what is this thing I am about to put in production*.

Introduction: The Capacity Plan That Was Off by a Factor of 199

A platform team is asked to self-host an open-weight 70-billion-parameter model. The reasoning takes about four minutes, and every step of it is defensible.

Seventy billion parameters at sixteen bits each is 141 gigabytes. Two 80 GiB accelerators give 160 GiB, so the weights fit with room to spare. The product wants 128,000-token contexts and sixty-four concurrent users. Two accelerators are provisioned. The load test is scheduled.

The load test never reaches sixty-four concurrent users. It never reaches one.

Here is what the arithmetic actually says, printed by the calculator you will build in section 9.14 of this chapter:

```
NAIVE PLAN
  weights bf16            :    131.4 GiB
  accelerator budget      :    160.0 GiB
  weights fit?            : yes
  therefore concurrency   : 64 assumed
  context per request     : 128,000 tokens
WHAT ACTUALLY HAPPENS
  free after weights+10%  :     12.6 GiB
  KV cache per sequence   :     39.1 GiB
  concurrent sequences    : 0
  memory needed for  64   :   2500.0 GiB (199x the free memory)
  outcome                 : requests 64 onwards queue or OOM
```

A single 128,000-token conversation needs 39.1 GiB of key-value cache — more than three times the memory left after the weights are loaded. Not sixty-four users. Not one. Zero.

The error was not a mistake in the multiplication. It was a mistake about **what a served model is**. The team modelled the deployment as *the weights*, because that is the number the model card advertises and the only number most discussions of model size mention. But a served language model is weights **plus a per-sequence state that grows with every token**, and at long context that state is the larger of the two.

This chapter is the anatomy lesson that prevents that plan. We will take model size apart until it stops being a single number and becomes a specification: what the parameters are, where they live, what they weigh at each precision, what else is loaded alongside them, what each user's conversation costs while it is open, and what any of it predicts about quality. By the end you will be able to look at a published architecture and say, with arithmetic rather than intuition, how many machines it needs and how many people it can serve at once.

One caution before we begin, and it matters for the exam as much as for the job. **Anthropic does not publish parameter counts, layer counts or architectural dimensions for Claude models.** Neither do most frontier labs. This is not an omission you can research your way around, and any source that states a parameter count for a Claude model is speculating. So the worked examples in this chapter use open-weight models whose configurations are published and whose totals can be checked — and every number in this chapter has been checked. What transfers to Claude is not the digits but the *structure*: the components, the ratios between them, the memory arithmetic, and above all the operational vocabulary in sections 9.10 to 9.12, which is what the certification actually tests.

9.1 What a Parameter Count Actually Counts

DEFINITION

Parameter, weight, bias, checkpoint

A **parameter** is a single number learned during training and frozen thereafter. Chapter 1 established that inference is forward-only and parameters do not change; Chapter 7 established that they are set by pre-training, then adjusted by supervised fine-tuning and reinforcement learning, and then frozen.

A **weight** is a parameter that multiplies something — an entry in one of the matrices of Chapter 4. A **bias** is a parameter that is added rather than multiplied. Most modern large language models omit biases in the attention and feed-forward projections entirely, so in practice a parameter count is a count of matrix entries.

A **checkpoint** is a file, or a set of files, containing the values of every parameter at a particular moment in training. "The model" that is served is one specific checkpoint.

Chapter 3 gave you the counting rule for a fully-connected layer: `(inputs × outputs) + outputs`. Everything in this chapter is that rule applied repeatedly and carefully. There is no other mathematics involved. A "70-billion-parameter model" is the sum of the entries in all its matrices — and, as Chapter 3 warned, that sum is a statement about **capacity and memory footprint**, not about quality.

Three things the number does not include are worth naming immediately, because each one has been the cause of a real capacity mistake.

It does not include the KV cache. The cache is not learned, so it is not a parameter. It is allocated at serve time, per sequence, and it is the subject of section 9.8.

It does not include activations. The intermediate values flowing through the network during a forward pass need memory too. For batch-1 decoding this is small; for large-batch prefill over long prompts it is not.

It does not include the optimiser state. Training a model requires storing gradients and optimiser moments — with the Adam optimiser of Chapter 3, typically two extra numbers per parameter. This is why training a model needs several times the memory of serving it, and why "can I run it?" has a different answer for training and inference.

9.2 The Anatomy of One Block

Chapter 4's section 4.8 gave a rough accounting: roughly $12 \times d^2$ parameters per block, with $4 \times d^2$ in attention and $8 \times d^2$ in the feed-forward network. That approximation is a good mental model and a bad calculator. Modern architectures differ from it in three specific ways, and each difference matters by billions of parameters.

Grouped-query attention shrinks the key and value projections. Chapter 5's section 5.7 showed that GQA gives several query heads a shared key/value pair. That saving applies to the *weights* as well as the cache: with 64 query heads and 8 key/value heads, `Wk` and `Wv` are one-eighth the size of `Wq` and `Wo`.

The SwiGLU feed-forward network has three matrices, not two. The classic transformer feed-forward block is `W1` then `W2` — expand, then contract. Most current models use a gated variant with three matrices: a *gate* projection and an *up* projection, both expanding, multiplied together elementwise, then a *down* projection contracting. The expansion factor is usually 3.5× rather than 4×, chosen so that

three matrices at 3.5× cost about the same as two at 5.25×.

The embedding matrices are large and are sometimes shared. A vocabulary of 128,256 tokens against a width of 8,192 is over a billion parameters for the input embedding alone, and the output projection back to vocabulary logits is the same size again. Some models *tie* these — using one matrix for both — halving the cost. Most large models do not.

Putting those together gives an exact accounting rather than an approximate one:

Component	Parameters	Notes
Input embedding	`vocab × d`	Token to vector, Chapter 6
Output projection	`vocab × d`	Vector to logits; zero if tied
Per block: `Wq`	`d × (n_heads × head_dim)`	Usually equals `d × d`
Per block: `Wk`, `Wv`	`d × (n_kv_heads × head_dim)` each	Shrunk by GQA
Per block: `Wo`	`(n_heads × head_dim) × d`	Usually equals `d × d`
Per block: feed-forward	`3 × d × ffn_hidden`	SwiGLU; `2 ×` for classic
Per block: norms	`2 × d`	Negligible, but counted
Final norm	`d`	Negligible, but counted

VISUAL ANALOGY

A parameter count is a floor area, not a business plan

A commercial letting agent describes a building as 70,000 square feet. That number tells you something real and useful: what it costs to heat, how many desks it holds, whether the lease is affordable. It tells you nothing at all about whether the company inside it is any good.

Parameter count is floor area. It bounds what can be stored and fixes what the deployment costs, and a bigger building can hold a bigger business. But a well-run 8,000-square-foot firm routinely outperforms a badly run 70,000-square-foot one, and knowing the square footage of two firms does not tell you which to hire.

Where the analogy breaks. Floor area is fixed by the building; a model's usable capacity is not fixed by its parameter count, because how well those parameters are used depends entirely on the training run that set them. Two models with identical dimensions and different training data are not two identical buildings — they are two entirely different companies. Worse, floor area is honest, whereas parameter counts are routinely quoted in incomparable ways: total versus active for mixture-of-experts models (section 9.5), with or without embeddings, before or after quantisation. Always ask which number you are being given.

9.3 A Real Model, Counted Exactly

Abstractions are cheap. Here is the calculator's output for Llama 3.1 70B — a model whose configuration is published, so every input is checkable and the total can be compared against the figure the model was released with.

```
=== Llama 3.1 70B ================================================
d_model 8192  layers 80  q_heads 64  kv_heads 8  head_dim 128
PARAMETERS BY COMPONENT
  embeddings         2,101,346,304      2.10B   2.98%
  attention         12,079,595,520     12.08B  17.12%
  feed_forward      56,371,445,760     56.37B  79.90%
  router                         0          0   0.00%
  norms                  1,318,912      1.32M   0.00%
  TOTAL             70,553,706,496     70.55B  100.00%
```

Read that table slowly, because four things in it are worth more than the total.

Eighty per cent of a large dense model is feed-forward. Attention gets all the attention — it is the mechanism the papers are about, the thing Chapter 4 spent twenty pages on, the part everyone can draw. It is 17% of the parameters. The feed-forward layers, which Chapter 4 described in two paragraphs, are 80%. If you have absorbed the idea that a transformer is "mostly attention", the arithmetic says otherwise: a transformer is mostly a stack of large fully-connected networks with an attention mechanism between them that decides what each one gets to look at.

Attention is cheap in weights and expensive in everything else. Those 12 billion attention parameters are responsible for the quadratic compute of section 4.10 and the entire KV cache of section 9.8. Parameter share and cost share are different things, and confusing them is a reliable way to optimise the wrong component.

The embeddings are 3% here and 13% in the 8B model. The embedding matrices scale with `vocab × d`, which is linear in width, while the blocks scale with d^2. So embeddings dominate small models and vanish in large ones. This is why vocabulary size is a serious design decision for a small model and a rounding error for a large one — and it interacts with the tokenizer economics of Chapter 6, where a larger vocabulary produces fewer tokens per document but a larger embedding table.

The norms are 1.3 million parameters out of 70.5 billion. Chapter 4's section 4.7 argued that normalisation is what makes deep stacks trainable at all. It is 0.002% of the model. Importance and size are unrelated.

The 70,553,706,496 figure is not a round number and should not be. Models are named for the nearest convenient magnitude — "70B" — and the actual count falls out of the dimensions. When you see a model advertised as 8B, the arithmetic gives 8,030,261,248. Both of those are exact matches for the published totals, which is how we know the calculator is right rather than merely plausible.

9.4 Width, Depth and the Two Directions of Scale

There are two ways to make a model bigger and they do not cost the same.

Widening increases `d_model`, and with it the head dimension and the feed-forward hidden size. Every matrix in every block has `d` on at least one side and usually both, so block parameters scale with d^2. Doubling the width quadruples the block parameters.

Deepening increases the number of blocks. Block parameters scale **linearly** in the layer count. Doubling the depth doubles the block parameters.

The calculator's test suite verifies exactly this, and the assertion is worth quoting because it is the whole of the argument:

```
assert blocks(wide) / blocks(base) == pytest.approx(4.0)   # 2x width
assert blocks(deep) / blocks(base) == pytest.approx(2.0)   # 2x depth
```

So why is any model wide at all, if depth is four times cheaper per unit of parameter?

Because the two buy different things, and Chapter 3's section 3.4 already explained the first half. Depth buys a **feature hierarchy**: each layer composes features detected by the layer below, and the number of layers bounds the length of the compositional chain the model can build. Width buys **representational bandwidth**: how much information the residual stream can carry from one layer to the next without one feature having to overwrite another.

There is also a hard engineering asymmetry that the parameter arithmetic hides. Layers are **sequential** — layer 41 cannot start until layer 40 finishes — so depth adds latency that no amount of hardware removes. Width is **parallel**: a wider matrix multiplication is a bigger job for the same GPU pass, exactly as Chapter 3's section 3.7 described. A very deep, narrow model and a shallow, wide model with the same parameter count have very different latency profiles, and the deep one is slower.

Real architectures therefore sit in a narrow band. Llama 3.1 70B is 80 layers at width 8,192 — a ratio of about 100 to 1 between width and depth. The 8B model is 32 layers at width 4,096, about 128 to 1. That band is not a law of nature; it is where the empirical scaling work of Chapter 12 says the compute-optimal trade-off sits, and it moves.

9.5 Mixture of Experts: Total Parameters and Active Parameters

If 80% of a model is feed-forward, and if only some of that capacity is relevant to any given token, an obvious question follows: must every token pay for all of it?

> DEFINITION
>
> **Mixture of experts, router, active parameters**
>
> A **mixture-of-experts (MoE)** model replaces the single feed-forward network in each block with several parallel copies, called **experts**. A small learned **router** — one matrix of shape `d × n_experts` — scores the experts for each token and sends the token to the highest-scoring few, typically two.
>
> **Total parameters** counts every expert. **Active parameters** counts only those a single token actually flows through. In a dense model the two are equal. In an MoE model the total can be several times the active count.

Mixtral 8x7B makes the gap concrete:

```
=== Mixtral 8x7B ================================================
PARAMETERS BY COMPONENT
  embeddings           262,144,000    262.14M   0.56%
  attention          1,342,177,280      1.34B   2.87%
  feed_forward      45,097,156,608     45.10B  96.56%
  router                 1,048,576      1.05M   0.00%
```

```
TOTAL              46,702,792,704     46.70B  100.00%
ACTIVE/token       12,879,925,248     12.88B  27.58%
```

Forty-seven billion parameters must be **in memory**. Twelve point nine billion are **used per token**. The router that makes the decision is a million parameters — 0.002% of the model — and it is the entire mechanism.

The operational consequences separate cleanly, and getting them the right way round is examinable.

Resource	Governed by	Why
Weight memory	**Total** parameters	Every expert must be resident; you cannot know in advance which token needs which
Compute per token (FLOPs)	**Active** parameters	Only the selected experts run
Decode latency	**Active** parameters, mostly	Fewer multiplications per token
Memory bandwidth	Somewhere between	Different tokens in a batch select different experts, so a batch may touch most experts anyway
Deployment cost	**Total** parameters	You are renting memory you must hold, not memory you use

Note the last row, because it is the trap. An MoE model gives you the quality of a large model at the compute cost of a smaller one — but the memory cost of the large one. If you are compute-bound, MoE is a large win. If you are memory-bound, which self-hosting long-context workloads usually are, it is not.

The name "8x7B" is itself a small lesson in reading claims carefully. Eight times seven is fifty-six; the model is 46.7B. The experts share the attention layers and embeddings, so the multiplication does not apply to the whole model. Do not compute a parameter count from a marketing name.

VISUAL ANALOGY

Experts are a hospital's specialist rota, not eight hospitals

A general hospital does not staff every ward with doctors capable of everything. A patient arrives, is triaged, and is routed to two relevant specialists out of eight on duty. The hospital pays all eight salaries whether or not they see a patient that hour — that is the total parameter count. Any individual patient consumes two specialists' time — that is the active count. And triage itself is one nurse, cheap and decisive, exactly like the router.

Where the analogy breaks. Hospital specialists correspond to human-legible specialisms — cardiology, orthopaedics — and it is tempting to assume MoE experts do too. They do not. Routing is learned end-to-end from the training objective, and inspection of trained routers generally finds groupings that follow syntactic and positional regularities rather than topics. There is no "French expert" or "Python expert" to point at. The second break: a hospital can send a patient to a third specialist if the first two are stumped, whereas the number of active experts is fixed by the architecture at every token, whether the token is hard or trivial. Chapter 13 covers the genuinely adaptive alternative — spending more *time* rather than more *experts* on hard inputs.

9.6 Precision: What a Number Weighs

A parameter is a number, and a number in a computer occupies a chosen number of bits. That choice is the cheapest large lever in the entire deployment.

DEFINITION

Precision formats

fp32 — 32-bit floating point, 4 bytes. The historical default for training. Rarely used for serving.

bf16 — "brain float", 16-bit, 2 bytes. Keeps fp32's exponent range but fewer mantissa bits, so it has the same dynamic range with less resolution. The de facto standard for serving.

fp8 — 8-bit floating point, 1 byte. Now well supported in hardware and increasingly used in production serving.

int4 — 4-bit integers, half a byte, with a scale factor shared across a block of weights. Aggressive; used when memory is the binding constraint.

Quantisation is the process of converting weights from a higher-precision format to a lower one.

The arithmetic is a multiplication, and the consequences are not:

```
WEIGHT MEMORY BY PRECISION  –  Llama 3.1 70B
  fp32    4.0 B/param     262.8 GiB   min accelerators @ 80 GiB: 4
  bf16    2.0 B/param     131.4 GiB   min accelerators @ 80 GiB: 2
  fp8     1.0 B/param      65.7 GiB   min accelerators @ 80 GiB: 1
  int4    0.5 B/param      32.9 GiB   min accelerators @ 80 GiB: 1
```

Each step down halves the memory. More importantly, each step crosses a **hardware boundary**: at bf16 this model needs two accelerators and therefore a network hop between them on every forward pass; at fp8 it needs one, and the hop disappears. The gain from quantisation is rarely the memory as such. It is the topology change the memory unlocks.

Quantisation is not free, and the costs are specific.

Quality degrades, unevenly. bf16 is effectively lossless in practice. fp8 typically costs very little. int4 costs measurably more, and — the part that matters — the loss is not distributed evenly across tasks. Long chains of arithmetic, tasks requiring precise recall of rare facts, and structured output formats tend to degrade first, while fluent conversational text holds up well. So a quantised model can look identical on a casual eyeball test and be materially worse on exactly the workload you deployed it for. **Evaluate a quantised model on your own task with the harness of Chapter 8. Do not accept a benchmark table.**

Not all weights are equally quantisable. Production quantisation schemes almost always keep some components at higher precision — commonly the embeddings, the output projection and the norms. This is why measured file sizes are larger than the naive multiplication predicts, and the calculator's figures are therefore a floor, not an exact prediction.

The KV cache is quantised separately. Weight precision and cache precision are independent choices. Section 9.8 shows this is often the more valuable of the two.

For the exam and for practice, note that none of this applies to the Claude API. You do not choose Claude's precision; Anthropic serves the model and prices the result. Quantisation is a self-hosting decision, and this section exists so that you can evaluate the *make* side of a make-versus-buy argument on numbers rather than vibes. Chapter 15 returns to that argument with the pricing on the other side.

9.7 The Served Model: Seven Components, Three You Control

Ask an engineer what is deployed when a language model is deployed and they will usually say "the weights". Here is what is actually running.

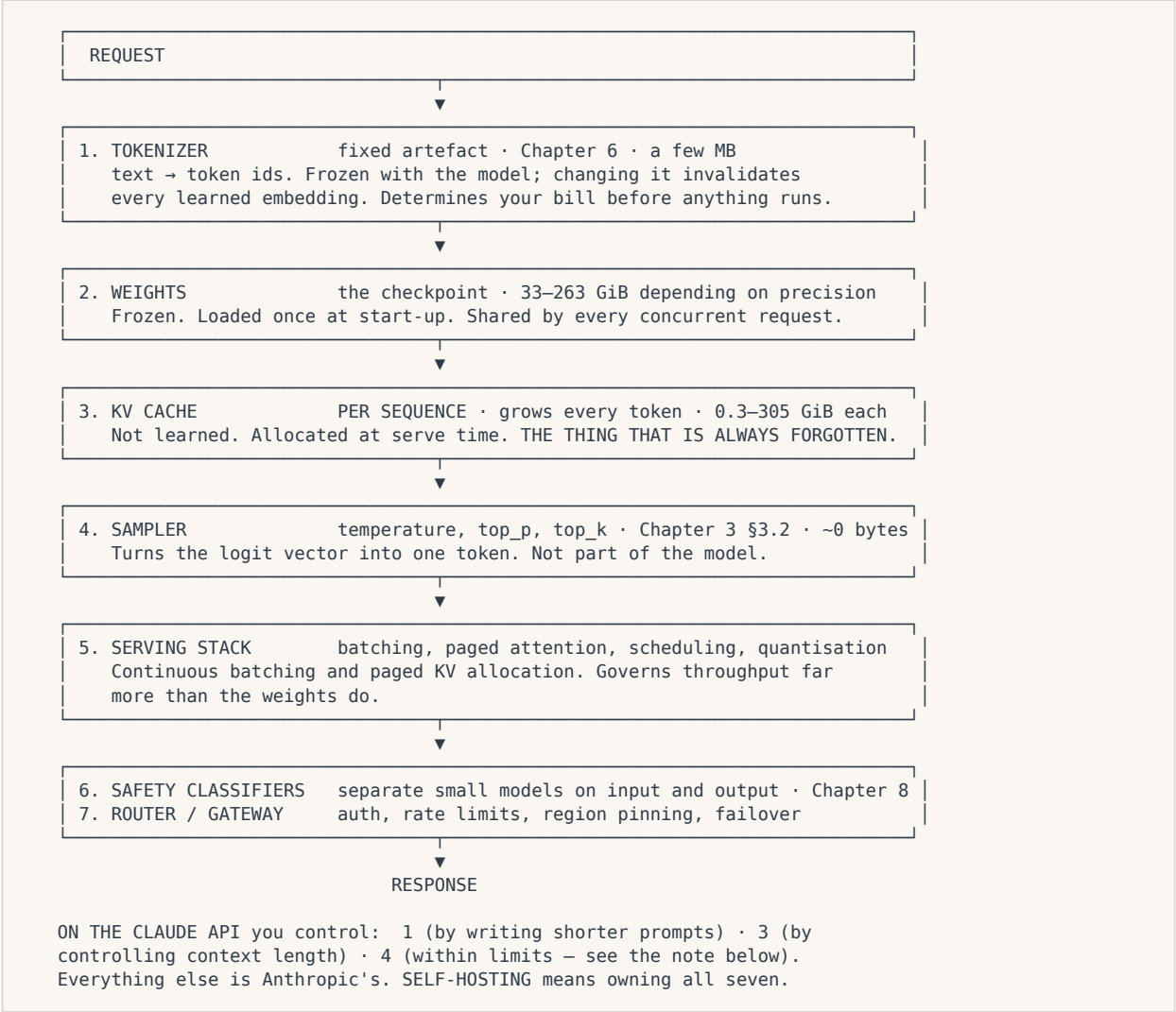


Figure 9.1 — The seven components of a served model. Three are yours.

That diagram is the practical thesis of the chapter. The parameter count is component 2 of seven, and it is the only one most conversations mention.

Two of the seven deserve a note now because they contradict common assumptions.

The sampler is not part of the model. Chapter 3's section 3.2 established that the model emits a probability distribution and something else chooses a token from it. That something else is a few lines of code with no parameters. It is why the same weights can be deterministic or wildly creative, and why "the model gave a different answer" is often a sampling fact rather than a model fact. On the Claude API

this control is deliberately narrow: according to the official documentation at the time of writing, passing non-default `temperature`, `top_p` or `top_k` to Claude Opus 5 or Claude Sonnet 5 returns a **400 error**. Chapter 27 covers what remains adjustable.

The serving stack changes behaviour without changing the model. This is documented rather than folklore. Anthropic's model versioning documentation states that model weights are fixed for a given model ID, but that the surrounding infrastructure — the request router, safety classifiers and sampling logic — can change over time, and that infrastructure updates occasionally produce minor observable behavioural differences on an unchanged model ID. Section 9.11 draws out what that means for your regression suite.

9.8 The KV Cache Is the Other Model

Chapter 4's section 4.9 introduced the KV cache as the reason decoding is fast. Chapter 5's section 5.7 gave the formula and three worked figures. This section states the consequence the two previous chapters deferred: **at long context, the per-sequence cache is comparable to or larger than the model itself, and it is the binding constraint on how many people you can serve.**

The formula, unchanged from section 5.7:

```
kv_bytes = 2 (K and V) × layers × kv_heads × head_dim × tokens × bytes_per_value
```

Everything on the right except `tokens` is fixed by the architecture. The cache is therefore **linear in context length**, and linear growth against a fixed memory budget is a hard ceiling rather than a gentle degradation.

```
   context      MHA          GQA          MQA        weights at bf16 = 131.4 GiB
               (64 kv)      (8 kv)       (1 kv)
  ──────────────────────────────────────────────────────────────────────────────
     8,000      19.53         2.44         0.31    ▒
    32,000      78.12         9.77         1.22    ▒▒
   128,000     312.50        39.06         4.88    ▒▒▒▒▒
   200,000     488.28        61.04         7.63    ▒▒▒▒▒▒▒
 1,000,000    2441.41       305.18        38.15    ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒

 Bars are scaled to the GQA column, which is the real architecture.

 READ THESE THREE FACTS OFF THE TABLE:
   · At 1M tokens the GQA cache (305 GiB) is 2.3x the entire model (131 GiB).
   · The same model built with plain MHA would need 2.4 TERABYTES per sequence.
     That is why every large model built after 2023 uses GQA. It is not a
     micro-optimisation; it is what made long context physically possible.
   · MQA is another 8x cheaper again and is used where quality permits.
```

Figure 9.2 — KV cache per sequence, Llama 3.1 70B at bf16 (GiB).

Now attach it to serving. Concurrency is what is left after the weights, divided by the cost per sequence:

```
concurrent sequences = (accelerator memory − overhead − weight memory)
                       ÷ (kv bytes per token × context length)
```

Run it on a full eight-accelerator node — 640 GiB, 10% reserved for activations and fragmentation:

```
CONCURRENCY on 8 x 80 GiB = 640 GiB, 10% reserved   —  Llama 3.1 70B
```

```
   context  w:bf16 kv:bf16   w:fp8 kv:bf16   w:fp8 kv:fp8
     8,000             182             209            418
    32,000              45              52            104
   128,000              11              13             26
 1,000,000               1               1              3
```

Four readings, each of which is worth an exam mark and a production incident.

Context length is a capacity decision, not a feature flag. Going from 8,000 to 128,000 tokens costs you sixteen-fold in concurrency — 182 users to 11 on identical hardware. Nobody experiences this as a capacity decision, because it arrives as a product request phrased as "let users upload bigger documents".

Quantising the cache beats quantising the weights. Weights fp8 alone takes 128K concurrency from 11 to 13. Cache fp8 as well takes it to 26. The weights are paid once and shared; the cache is paid per user. At long context, per-user costs dominate — which is the same shape of argument as Chapter 4's case against context-stuffing, arriving from the memory side rather than the compute side.

This is why long context is priced and rate-limited the way it is. Chapter 5 asserted that long-context serving is a memory problem before it is a compute problem. This is the table behind that sentence. When Chapter 19 covers rate limits, and Chapter 30 covers cost controls, the underlying physics is here.

Retrieval is a capacity strategy, not only a quality strategy. Chapter 5's two-stage pipeline and Chapter 53's RAG architecture are usually justified by relevance and attention dilution. Add a second justification with a number on it: cutting context from 128K to 32K in the naive-plan scenario took concurrency from 0 to 16 on unchanged hardware. Retrieval is the cheapest capacity upgrade available, and it requires no new machines.

VISUAL ANALOGY

The weights are the library; the cache is each reader's desk

A reference library buys one copy of each book. Cost paid once, shared by everyone — that is the weights. Every reader who walks in also needs a desk, and the desk is piled higher with every page they consult; a reader working through a thousand documents needs an enormous desk. That is the KV cache.

The library's capacity is not set by its books. It is set by its floor space minus the shelves, divided by the average desk size. Doubling the depth of everyone's research does not require more books. It requires more floor, and there isn't any.

Where the analogy breaks. Desks can be cleared and shared; a reader who steps out frees their space. KV cache for an *active* sequence cannot be freed — every entry is needed for the next token, by causal masking (Chapter 4, section 4.4). Modern serving stacks do recover the space between requests and can page cold blocks out, but within a generating sequence the cache only grows. And unlike desks, cache size is not a matter of tidiness: it is exactly determined by token count. There is no efficient reader.

9.9 What Model Size Does and Does Not Predict

Chapter 3 drew a line between **capacity** — what a network could in principle represent — and **capability** — what this particular trained network actually does. Parameter count is a statement about the first. Here is a table of what it therefore does and does not tell you.

Parameter count predicts	Parameter count does not predict
Weight memory, exactly	Quality on your task
Minimum accelerators to hold the weights	Instruction-following or output-format reliability
Roughly, the compute per token (for dense models)	Tool-use competence or agentic reliability
Roughly, the floor on decode latency	Refusal behaviour and calibration
Deployment cost if you are self-hosting	Context window length
The order of magnitude of training cost	Long-context retrieval quality

The right-hand column is not a list of things that are merely hard to predict. It is a list of things determined by **the training run**, not the architecture: by data quality and mixture, by the post-training of Chapter 7, by how much compute was spent and on what. Two models with identical dimensions and different training are not similar models.

Three specific inferences are wrong often enough to name.

"Bigger model, longer context." Context length is a training and serving decision, essentially independent of parameter count. Anthropic's documentation records Claude Haiku 4.5 at a 200K-token context window while Claude Opus 5, Sonnet 5 and Fable 5 are at 1M — and the models are not ordered by size in any way you can verify. Small models with long contexts and large models with short ones both exist.

"Bigger model, better at everything." Capability gains from scale are real but uneven. Frontier-scale models are dramatically better at multi-step reasoning, code and long-horizon agentic work; the gap on classification, extraction and formatting is often small enough that a smaller model wins on cost and latency. Chapter 15 turns this into a selection procedure.

"Parameter count is comparable across models." It is not, and section 9.5 showed why: total versus active for MoE, with or without embeddings, and different training budgets behind identical dimensions. Chapter 12's scaling laws explain why a 70B model trained on 15 trillion tokens beats a 70B model trained on 300 billion by a margin that no architectural difference could produce.

For Claude specifically, the sizes are not published, so none of these inferences is even available to you. What you get instead is a **published capability and price ordering** — the model comparison table of Chapter 14 — plus the evaluation methodology of Chapter 8 to test it against your own workload. That is a better basis for a decision than a parameter count would have been.

9.10 Model, Checkpoint, Deployment, Alias

Four words are used interchangeably in conversation and mean four different things in an incident review. The distinctions in this section are directly examinable and are the most operationally consequential material in the chapter.

DEFINITION

The four levels

A **model** is an architecture plus a training recipe — "Claude Opus 5". It is the name of a research and engineering artefact, and it is what marketing talks about.

A **checkpoint** is one specific set of frozen parameter values produced by that recipe. Training produces many; one is selected, evaluated and released.

A **deployment** is a checkpoint plus everything in Figure 9.1 around it — the serving stack, the sampler configuration, the safety classifiers, the routing layer, in a particular region.

An **alias** is a name that resolves to a checkpoint. It may be stable forever or it may move.

Anthropic's documented versioning scheme maps onto these cleanly, and it changed in a way that catches people out.

According to the official documentation at the time of writing:

- Every Claude model ID identifies a **pinned version**. When you use a model ID, the underlying model remains constant for the lifetime of that ID.
- Models **before the 4.6 generation** carry a snapshot date in the ID — `claude-haiku-4-5-20251001`, `claude-sonnet-4-5-20250929`. For these, the Claude API also accepts a shorter **alias** (`claude-sonnet-4-5`) that is a convenience pointer resolving to the most recent dated snapshot for that minor version.
- Starting with the **4.6 generation**, model IDs use a **dateless** format — `claude-opus-5`, `claude-sonnet-5`, `claude-opus-4-8`. The documentation is explicit that these are **not** evergreen pointers: a dateless ID for the 4.6 generation onward is the canonical, pinned snapshot itself. Anthropic does not update the weights or configuration of an existing model ID; an updated version ships under a new ID.
- Every model ID, dated or dateless, has its own deprecation and retirement schedule.

The misconception the documentation exists to correct is worth stating plainly, because it is precisely backwards from what the naming suggests. `claude-sonnet-4-5` **is** a moving alias. `claude-sonnet-5` **is not** — it looks like an alias and is a pin.

You write	Generation	What it is	What can change under you
`claude-haiku-4-5-20251001`	Pre-4.6	Dated snapshot	Nothing (weights); serving infrastructure can
`claude-sonnet-4-5`	Pre-4.6	Convenience alias	Resolves to a newer dated snapshot
`claude-opus-5`	4.6+	Pinned snapshot	Nothing (weights); serving infrastructure can

WARNING

What "pinned" does not mean

Pinned means the **weights** are fixed. It does not mean behaviour is frozen.

Anthropic's versioning documentation states that the serving infrastructure around the model — request router, safety classifiers, sampling logic — can change over time, and that infrastructure updates occasionally produce minor differences in observable behaviour even when the model ID and weights have not changed. It goes on to say that if you observe unexpected behavioural differences on a previously stable model ID, an infrastructure update is the most likely cause.

The architectural consequence is unambiguous: **pinning a model ID reduces drift; it does not eliminate it.** Your regression suite from Chapter 24 runs on a schedule, not only at upgrade time. An architect who believes a pin makes evaluation unnecessary has misread the guarantee.

The operational discipline follows from the table above:

Pin explicitly in production. Write the full model ID in configuration, not a name you assume is stable. If you are on a pre-4.6 model, use the dated form.

Make the model ID a deployable value. It belongs in configuration you can change without a code release, because you will need to change it under time pressure at some point.

Treat a model ID change as a code change. Same review, same evaluation gate, same rollback plan. It is the largest single-line change you can make to a system's behaviour.

Watch the deprecation schedule per ID. Each has its own, and retirement dates are published. Discovering one from a 404 is avoidable.

9.11 What Actually Changes Between Generations

If the weights are opaque, what should you actually track when a new model appears? Six things, in rough order of how likely they are to break you.

The tokenizer. Chapter 6 verified this and it remains the sharpest edge in the list: Claude 4.7 and later use a newer tokenizer that produces roughly **30% more tokens** for the same text than earlier models, and billing reflects it. A cost model calibrated before that boundary is wrong after it. Never reuse token counts across a tokenizer change; re-measure with `count_tokens`, which is free.

The context window and maximum output. According to the official documentation at the time of writing, Claude Fable 5, Opus 5 and Sonnet 5 have a 1M-token context window with 128K maximum output, and Claude Haiku 4.5 has 200K with 64K maximum output. For 1M-context models, 1M is the default: no beta header, and standard pricing. The documentation also notes that the Message Batches API supports up to 300K output tokens on several models via a beta header.

Which parameters are accepted. The sampling restriction of section 9.7 is a generation-level behaviour, not a permanent property of the API. Re-read the parameter documentation on every upgrade; a 400 error in production is an expensive way to discover a change.

Thinking behaviour. The documentation distinguishes *extended thinking* (`thinking.type: "enabled"`) from *adaptive thinking*, and the current table shows Fable 5, Opus 5 and Sonnet 5 supporting adaptive

thinking while Haiku 4.5 supports extended thinking. Whether previous thinking blocks are retained in context also varies by model. Chapter 18 covers this in full; the point here is that it changes between generations and affects both cost and context consumption.

Prompt caching thresholds. The minimum cacheable prompt on Claude Opus 5 is 512 tokens, reduced from 1,024 on Opus 4.8. A prompt engineered to sit just above an old threshold may fall below a new one, or vice versa.

Prompt sensitivity. The least tractable item. Prompts tuned against one checkpoint's quirks may perform differently against another — usually better, occasionally not. This is the reason the regression suite of Chapter 24 exists and the reason Chapter 20 argues for prompts that state intent rather than exploit idiosyncrasies.

Notice what is *not* on the list: parameter count, layer count, precision, architecture. Not because they do not matter to the model's quality — they matter enormously — but because they are neither published nor actionable. **You cannot manage what you are not told, so manage what you are told.** The published surface is the model ID, the context window, the maximum output, the accepted parameters, the pricing and the tokenizer. That surface is your contract, and it is a complete one for architectural purposes.

9.12 Where the Anatomy Sits in Your Architecture

Pulling the chapter together into the shape of a decision:

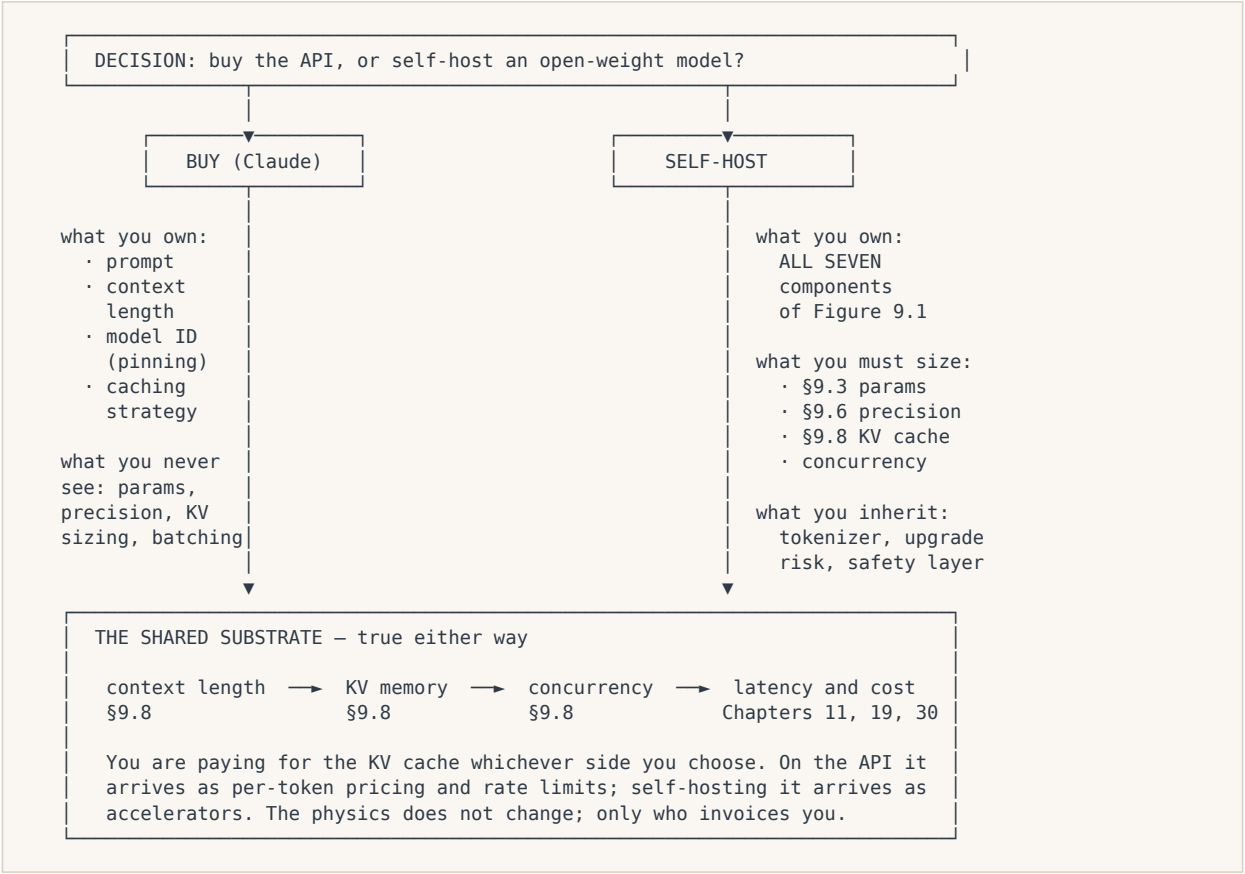


Figure 9.3 — Where each part of the anatomy enters an architectural decision.

The last box is the sentence to carry out of this chapter. The reason the Claude API prices long context the way it does, and rate-limits it the way it does, is the table in section 9.8. Understanding the anatomy does not let you change Anthropic's serving stack. It lets you predict its behaviour, argue about its costs with numbers, and recognise which of your own design choices are the expensive ones.

9.13 Code Walkthrough: Counting a Model From Its Specification

Everything in sections 9.3 to 9.8 came out of one small module. It is worth reading line by line, because the whole of model anatomy is in about eighty lines of arithmetic and there is nothing hidden in it.

Start with the specification itself. A dataclass is the right shape here: an architecture *is* a bag of numbers, and giving each one a name is most of the education.

```
@dataclass
class Arch:
    """A decoder-only transformer architecture specification."""
    name: str
    vocab: int              # tokenizer vocabulary size
    d_model: int            # residual stream width
    n_layers: int           # number of transformer blocks
    n_heads: int            # query heads
    n_kv_heads: int         # key/value heads (== n_heads for MHA, 1 for MQA)
    head_dim: int           # per-head width
    ffn_hidden: int         # feed-forward inner width
    ffn_matrices: int = 3   # 3 for SwiGLU (gate, up, down); 2 for classic
    n_experts: int = 1      # 1 = dense; >1 = mixture-of-experts
    n_active_experts: int = 1
    tied_embeddings: bool = False   # share input and output embedding matrix
    norms_per_layer: int = 2        # RMSNorm vectors of length d_model
```

Line by line, the ones that carry weight:

`n_heads` **and** `n_kv_heads` **are separate fields.** This is the single most important line in the class. Chapter 5's Figure 5.3 showed them diverging; here that divergence becomes two independent numbers, and every GQA saving in the chapter falls out of the gap between them. Setting them equal gives MHA. Setting `n_kv_heads = 1` gives MQA.

`head_dim` **is not derived from** `d_model / n_heads`**.** It usually equals it, but not always, and hard-coding the division silently breaks on models where it does not hold.

`ffn_matrices` **defaults to 3.** The classic transformer has two feed-forward matrices; SwiGLU has three. Defaulting to 2, as most naive calculators do, understates a modern model's feed-forward parameters by a third — which, given that feed-forward is 80% of the model, understates the total by about 27%.

`tied_embeddings` **defaults to** `False`**.** Getting this wrong changes an 8B model's total by 525 million parameters. It is worth a boolean.

Now the attention accounting:

```
    def attention_params_per_layer(self) -> int:
        q = self.d_model * self.n_heads * self.head_dim
        k = self.d_model * self.n_kv_heads * self.head_dim
```

```
        v = k
        o = self.n_heads * self.head_dim * self.d_model
        return q + k + v + o
```

Four matrices, exactly as in Chapter 4's section 4.2.2: `Wq`, `Wk`, `Wv`, `Wo`. The asymmetry is the whole point — `q` and `o` use `n_heads`, while `k` and `v` use `n_kv_heads`. For Llama 3.1 70B that is 64 against 8, so the key and value projections together cost a quarter of what the query projection alone does. Chapter 5 told you GQA shrinks the cache. It also shrinks the weights, which is a second saving rarely mentioned.

The feed-forward side splits into per-expert and per-layer, so that dense and MoE models go through the same code path:

```
    def ffn_params_per_expert(self) -> int:
        return self.ffn_matrices * self.d_model * self.ffn_hidden
    def ffn_params_per_layer(self) -> int:
        return self.n_experts * self.ffn_params_per_expert()
    def router_params_per_layer(self) -> int:
        return self.d_model * self.n_experts if self.n_experts > 1 else 0
```

A dense model is an MoE model with one expert and no router. Writing it that way rather than branching on a flag means there is one accounting to get right instead of two, and the dense case is checked by the same tests as the sparse one.

The breakdown assembles the components and — importantly — asserts nothing about their sum. It returns them, and a test checks that the parts add up:

```
    def breakdown(self) -> dict:
        """Total parameters, by component. Keys sum to 'total'."""
        attn = self.attention_params_per_layer() * self.n_layers
        ffn = self.ffn_params_per_layer() * self.n_layers
        router = self.router_params_per_layer() * self.n_layers
        norms = self.norm_params_per_layer() * self.n_layers + self.d_model
        emb = self.embedding_params()
        out = {"embeddings": emb, "attention": attn, "feed_forward": ffn,
               "router": router, "norms": norms}
        out["total"] = sum(out.values())
        return out
```

Active parameters are where MoE stops being a footnote:

```
    def active_params(self) -> int:
        b = self.breakdown()
        if self.n_experts == 1:
            return b["total"]
        active_ffn = (self.n_active_experts * self.ffn_params_per_expert()
                      * self.n_layers)
        return (b["embeddings"] + b["attention"] + active_ffn
                + b["router"] + b["norms"])
```

Every component except the feed-forward experts appears in full. Embeddings, attention, router and norms are used by every token regardless of routing; only the experts are selective. This is exactly why the total-to-active ratio of Mixtral is 3.63 rather than 4.0 despite selecting 2 of 8 experts — the shared components dilute the saving.

Then memory, which is a multiplication and a lookup:

```
BYTES_PER_PARAM = {"fp32": 4.0, "bf16": 2.0, "fp8": 1.0, "int4": 0.5}
    def weight_bytes(self, precision: str = "bf16") -> float:
        return self.breakdown()["total"] * BYTES_PER_PARAM[precision]
    def kv_bytes_per_token(self, precision: str = "bf16") -> float:
        """2 (K and V) x layers x kv_heads x head_dim x bytes."""
        return (2 * self.n_layers * self.n_kv_heads * self.head_dim
                * BYTES_PER_PARAM[precision])
```

`kv_bytes_per_token` deliberately does not take a token count. Separating the per-token constant from the multiplication makes the linearity visible in the code as well as in the output, and it is the number you actually want when reasoning about a workload — bytes per token is a property of the model, not of the request.

Finally the number that would have saved the team in the introduction:

```
    def concurrency(self, accel_gib: float, context: int,
                    weight_precision: str = "bf16",
                    kv_precision: str = "bf16",
                    overhead_gib: float = 0.0) -> int:
        """How many sequences of `context` tokens fit alongside the weights."""
        free = (accel_gib - overhead_gib) * GIB - self.weight_bytes(
            weight_precision)
        if free <= 0:
            return 0
        return int(free // self.kv_bytes(context, kv_precision))
```

Three details make this honest rather than optimistic. `weight_precision` and `kv_precision` are **separate arguments**, because they are separate decisions and section 9.8 showed the second matters more. `overhead_gib` is explicit rather than hidden in a fudge factor, so the assumption is visible in the call site. And the function returns `0` rather than a negative number when the weights do not fit — floor division on a negative numerator produces a plausible-looking wrong answer, which is exactly the failure mode this whole module exists to prevent.

9.14 Mini Project: The Anatomy Calculator

What you are building. A tool that takes an architecture specification and answers the four questions a capacity plan needs: how many parameters, how much weight memory at each precision, how much KV cache per sequence against context length under each attention variant, and how many concurrent users a given accelerator budget supports. Then a second script that demonstrates the naive plan failing, and shows which lever fixes it.

Why it is worth building rather than reading. Because the numbers are checkable. Any tool that reports a model's parameter count can be validated against the published total, and a calculator that reproduces three real models exactly is a calculator you can trust on a fourth. This is the same discipline as Chapter 6's from-scratch BPE benchmarked against the `tokenizers` library: build it, then prove it.

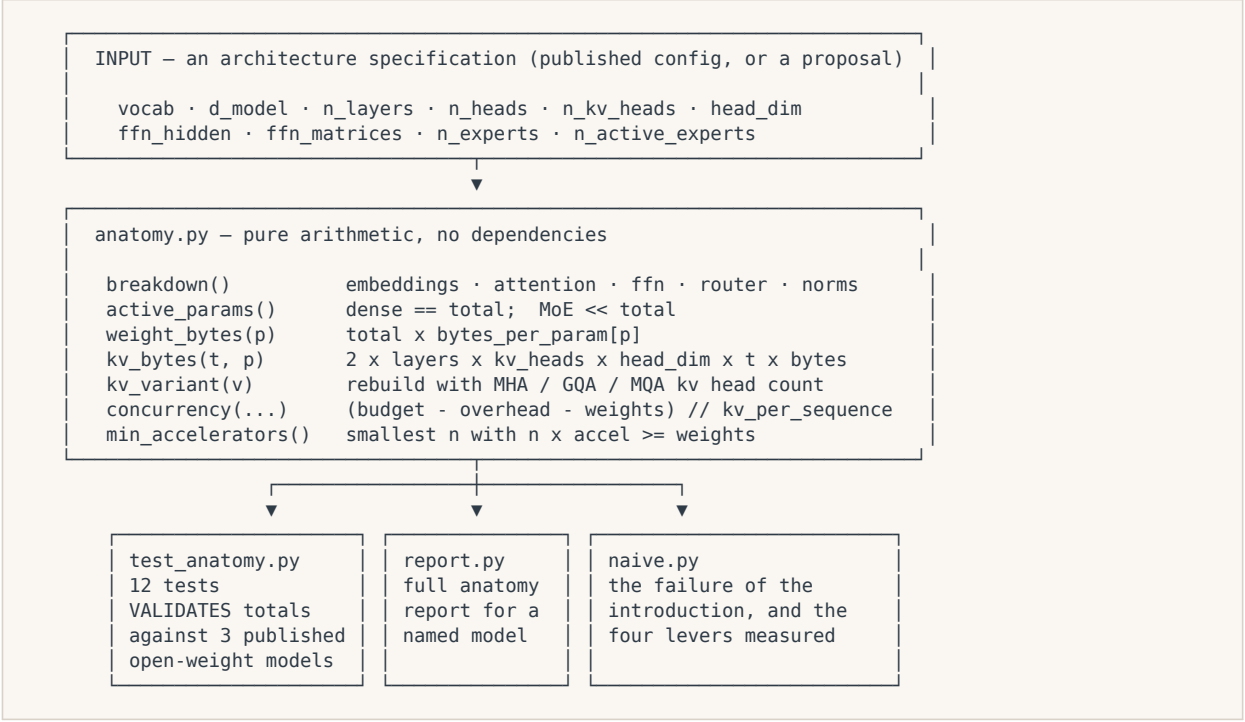


Figure 9.4 — The anatomy calculator.

Folder structure.

```
ch9/
  anatomy.py         the Arch dataclass and all arithmetic; no imports
  report.py          formatted anatomy report for a named architecture
  naive.py           the introduction's failure, reproduced and repaired
  test_anatomy.py    12 tests, including validation against published totals
```

`anatomy.py` was walked through in section 9.13 and is reproduced in full in the source bundle. Here is `report.py`'s core, which is only formatting but is where the design decision about *what to show* lives:

```
def report(arch: Arch, accel_gib: float = 80.0, n_accel: int = 8) -> None:
    b = arch.breakdown()
    total = b["total"]
    print("PARAMETERS BY COMPONENT")
    for k in ("embeddings", "attention", "feed_forward", "router", "norms"):
        pct = 100.0 * b[k] / total
        print(f"  {k:<14} {b[k]:>17,}  {human(b[k]):>9}  {pct:5.2f}%")
    print(f"  {'TOTAL':<14} {total:>17,}  {human(total):>9}  100.00%")
    if arch.n_experts > 1:
        act = arch.active_params()
        print(f"  {'ACTIVE/token':<14} {act:>17,}  {human(act):>9}  "
              f"{100.0 * act / total:5.2f}%")
```

Percentages are printed alongside absolute counts deliberately. The absolute number tells you what to buy; the percentage tells you what to optimise. A report that gives only one of them produces engineers who can size a cluster but not improve it, or the reverse.

The KV table sweeps both context length and attention variant, because the interesting fact is the interaction between them:

```
    print("KV CACHE PER SEQUENCE, bf16 (GiB)")
    print(f"  {'context':>9} {'MHA':>10} {'GQA':>10} {'MQA':>10}")
    for ctx in (8_000, 32_000, 128_000, 200_000, 1_000_000):
        row = [arch.kv_variant(v).kv_bytes(ctx) / GIB
               for v in ("MHA", "GQA", "MQA")]
        print(f"  {ctx:>9,} {row[0]:>10.2f} {row[1]:>10.2f} {row[2]:>10.2f}")
```

The instructive failure. `naive.py` reproduces the reasoning from the chapter's introduction and then measures it. Its structure is the point: the naive planner never computes a per-sequence cost at all, because it does not know there is one.

```
def naive_plan(target_concurrency=64, context=128_000):
    w = M.weight_bytes("bf16")
    print("NAIVE PLAN")
    print(f"  weights bf16            : {w / GIB:8.1f} GiB")
    print(f"  accelerator budget      : {BUDGET:8.1f} GiB")
    print(f"  weights fit?            : {'yes' if w < BUDGET * GIB else 'no'}")
    print(f"  therefore concurrency   : {target_concurrency} assumed")
def actual(target, context):
    w = M.weight_bytes("bf16")
    free = BUDGET * (1 - RESERVE) * GIB - w
    per = M.kv_bytes(context, "bf16")
    fits = int(free // per) if free > 0 else 0
    print(f"  KV cache per sequence   : {per / GIB:8.1f} GiB")
    print(f"  concurrent sequences    : {fits}")
```

Running it produces the output quoted in the introduction: 39.1 GiB of cache per sequence against 12.6 GiB free, and a concurrency of zero. The second half of the script measures the four available fixes on the same two-accelerator budget:

```
THE FOUR LEVERS, MEASURED
  baseline  bf16 weights, bf16 cache, 128K ctx    0 concurrent
  quantise weights to fp8                         2 concurrent
  quantise the KV cache to fp8 as well            4 concurrent
  cut context to 32K (retrieval, not stuffing)   16 concurrent
```

Read the levers in order of return. Quantising the weights unblocks the deployment but barely improves it — it frees a fixed 65.7 GiB once. Quantising the cache doubles what remains, because it halves a *per-user* cost. Cutting context from 128K to 32K quadruples it again, because it halves that per-user cost twice more. Zero to sixteen, on the same two accelerators, without buying anything.

That ordering generalises, and it is the most portable result in the chapter: **at long context, attack per-sequence costs before shared costs.** The same logic drives the retrieval architecture of Chapter 53 and the compaction strategies of Chapter 54, and it is the memory-side twin of the compute-side argument Chapter 4 made against context-stuffing.

Testing. Twelve tests, of which four are validations against external ground truth and the rest are properties that must hold for any architecture.

```
def test_llama_70b_total_matches_published():
    # Published total for Llama 3.1 70B: 70,553,706,496 parameters.
    assert LLAMA31_70B.breakdown()["total"] == 70_553_706_496
def test_mixtral_total_and_active_match_published():
    # Mixtral 8x7B is published as 46.7B total / 12.9B active.
```

```
    assert MIXTRAL_8X7B.breakdown()["total"] == 46_702_792_704
    assert MIXTRAL_8X7B.active_params() == 12_879_925_248
def test_gqa_saves_exactly_the_head_ratio():
    mha = LLAMA31_70B.kv_variant("MHA").kv_bytes_per_token()
    gqa = LLAMA31_70B.kv_variant("GQA").kv_bytes_per_token()
    mqa = LLAMA31_70B.kv_variant("MQA").kv_bytes_per_token()
    assert mha / gqa == 64 / 8          # 8x saving
    assert mha / mqa == 64 / 1          # 64x saving
def test_kv_cache_overtakes_weights_at_long_context():
    a = LLAMA31_70B
    weights = a.weight_bytes("bf16")
    cache = a.kv_bytes(1_000_000, "bf16")
    assert 0.5 * weights < cache < 3.0 * weights
    assert a.kv_variant("MHA").kv_bytes(1_000_000) > 10 * weights
```

Run them:

```
$ cd ch9 && python3 -m pytest -q
............                                                             [100%]
12 passed in 0.03s
```

The exact-equality assertions against published totals are what make this more than a plausible-looking spreadsheet. Three independent architectures — dense small, dense large, sparse — all reproduce to the parameter. A calculator that hits `70,553,706,496` on the nose is not approximately modelling something; it is doing the same arithmetic the model does.

Deployment. The module has no dependencies beyond the standard library, so shipping it is copying one file. In practice it belongs in two places. First, in your capacity-planning repository, invoked in CI whenever someone proposes a context-length increase, with a test that fails if projected concurrency drops below the service level. Second — and this is the higher-value use — as an argument-settling tool in design review. "We can support 128K contexts" and "we can support 128K contexts for eleven concurrent users on a full node" are different sentences, and only one of them can be planned around.

Evaluation. Judge the calculator on three criteria, in order:

1. **Correctness against published totals.** Non-negotiable, and easy: add a fourth open-weight model from its config file and check the total. If it does not match, your understanding of that architecture is wrong and the tool has told you so.
2. **Honesty about what it excludes.** It models weights and KV cache. It does not model activation memory, fragmentation, framework overhead or the serving stack's own allocations, which is why every call reserves 10%. A capacity tool that quietly omits a term is worse than no tool, because it is trusted.
3. **Whether it changes a decision.** The measure of a planning tool is not accuracy in the abstract but whether anyone acts on it. If the 128K-versus-32K row does not come up in your next design review, the tool is not deployed — it is merely written.

9.15 Hands-on Labs

Each lab has an explicit success criterion. Work in `ch9/`.

Lab 9.1 — Count a fourth model. Choose any open-weight model whose configuration is published, transcribe its dimensions into an `Arch`, and compare your total against the published parameter count. *Success criterion: your total matches the published figure exactly, or you can name the specific architectural feature (tied embeddings, biases, an unusual feed-forward, a vision tower) that accounts for the difference.*

Lab 9.2 — Find the crossover point. For Llama 3.1 70B at bf16, find the context length at which the KV cache for a single sequence equals the weight memory. *Success criterion: you report a token count and can state, in one sentence, why every model built for long context uses GQA.*

Lab 9.3 — Size a real deployment. You must serve 200 concurrent users at 32,000 tokens of context with a 70B-class model. Using `concurrency()` and `min_accelerators()`, find the cheapest configuration of 80 GiB accelerators, considering both weight and cache precision. *Success criterion: an accelerator count, a precision pair, and a sentence saying which quality risk you have accepted.*

Lab 9.4 — Build the MoE trade-off table. For Mixtral 8x7B and a hypothetical dense model with the same *active* parameter count, compare weight memory, KV cache and concurrency at 32K context. *Success criterion: a table showing that MoE costs more memory and the same compute, and a sentence saying when that trade is worth making.*

Lab 9.5 — Break the calculator honestly. Add a `tied_embeddings=True` model and a `ffn_matrices=2` model. Predict the parameter change before running. *Success criterion: your prediction is within 1% of the computed figure, and you can state which real architectures use each option.*

9.16 Lab Solutions

9.1. Llama 3.1 8B: `vocab=128256, d_model=4096, n_layers=32, n_heads=32, n_kv_heads=8, head_dim=128, ffn_hidden=14336`. The calculator returns 8,030,261,248, matching the published total exactly. The commonest source of a mismatch is `tied_embeddings`: a model that ties them and a model that does not differ by `vocab × d`, which for this architecture is 525,336,576 parameters — 6.5% of the model, more than enough to look like a bug in your arithmetic when it is a fact about the model.

9.2. The per-token GQA cache is `2 × 80 × 8 × 128 × 2 = 327,680` bytes. Weight memory at bf16 is 131.4 GiB, or 141,107,412,992 bytes. Dividing gives **430,625 tokens** — under half the 1M-token window the model class targets. Under MHA the same crossover arrives at 53,828 tokens, which is barely a long document. That is the sentence: without GQA, the cache for a single ordinary-length conversation would exceed the entire model, and long-context serving at any useful concurrency would be arithmetically impossible rather than merely expensive.

9.3. From the concurrency table, one 640 GiB node at 32K context supports 45 sequences at bf16/bf16, 52 at fp8/bf16, and 104 at fp8/fp8. Two hundred users therefore need two nodes at fp8/fp8 (208 capacity), or five nodes at bf16/bf16 (225). The fp8 route is 2.5× cheaper in hardware and carries the risk named in section 9.6: **quantisation degrades unevenly**, and a KV cache at fp8 degrades long-context recall specifically — precisely the capability you provisioned long context to provide. The defensible answer is two fp8/fp8 nodes *plus* a Chapter 8 evaluation on long-context retrieval comparing fp8 against bf16 on your own data. If the evaluation shows a regression, you have a measured reason to spend the money.

9.4. Mixtral 8x7B at 32K context: 87.0 GiB of weights, 3.91 GiB of cache per sequence, 125 concurrent on a 640 GiB node. A hypothetical dense 12.9B model with the same architecture shape would hold 24.0 GiB of weights and — since the attention configuration is identical — the same 3.91 GiB of cache, giving 141 concurrent. So the MoE model costs 63 GiB more memory and 16 concurrent sequences for the same compute per token. The trade is worth making when the extra experts buy quality you would otherwise have to reach for with a much larger dense model, and it is not worth making when you are memory-bound — which, at long context, you almost always are.

9.5. Tying embeddings on Llama 3.1 8B removes `128256 × 4096 = 525,336,576` parameters, a 6.5% reduction to 7,504,924,672. Changing `ffn_matrices` from 3 to 2 removes one-third of 5,637,144,576, or 1,879,048,192 — a 23.4% reduction. Tied embeddings are common in small models where the embedding table is a large share; three-matrix SwiGLU is near-universal in current large models. The lesson is that a calculator using the classic two-matrix feed-forward understates a modern model by roughly a quarter, which is why section 9.13 made that default explicit.

9.17 Best Practices

BEST PRACTICE

Working with model anatomy

1. **Compute the KV cache before you promise a context length.** Every context-length commitment is a concurrency commitment, and the conversion is one multiplication. Do it in the design review, not the load test.
2. **Quote total and active parameters separately** for any MoE model, and say which one you mean when you say "size". Memory follows total; compute follows active.
3. **Pin model IDs explicitly in production configuration**, and put the ID somewhere you can change without a code release.
4. **Treat a model ID change as a code change** — same review, same evaluation gate, same rollback plan.
5. **Re-measure token counts after any tokenizer change.** `count_tokens` is free and rate-limited independently of the Messages API; there is no excuse for a stale cost model.
6. **Evaluate quantised models on your own workload**, not on published benchmarks. Degradation is uneven and lands hardest on precise recall and structured output.
7. **Reserve at least 10% of accelerator memory** for activations, fragmentation and framework overhead before computing concurrency.
8. **Attack per-sequence costs before shared costs** when memory-bound. Cache precision and context length beat weight precision, usually by a lot.
9. **Prefer retrieval to a longer window** — it is a capacity upgrade as well as a quality one, and it requires no new hardware.
10. **Keep a regression suite running on a schedule**, not only at upgrade time, because pinning fixes the weights and not the serving stack.
11. **Never infer capability from parameter count.** Use the evaluation methodology of Chapter 8 on your own task.
12. **Record the exact model ID in every log line** that captures a model output. Post-incident analysis without it is guesswork.
13. **Track deprecation dates per model ID**, since each ID carries its own schedule.
14. **Write down the assumptions in any capacity model**, especially what it excludes. An unstated exclusion becomes an assumed inclusion.

9.18 Common Mistakes

COMMON MISTAKE

Sizing a deployment on weight memory alone

Symptom. The model loads, single-request tests pass, and the service collapses or queues indefinitely the moment real concurrency arrives. Memory graphs show a near-full accelerator with only a handful of active requests.

Cause. The capacity plan modelled the weights and omitted the KV cache. At 128K context on a 70B GQA model, each sequence needs 39.1 GiB — more than the free memory after loading the weights on two accelerators.

Fix. Compute `weights + concurrency × kv_bytes(context)` and size against that. Reserve 10% for overhead.

Detection. A load test at your *maximum* supported context length and target concurrency, not at a typical one. Single-request benchmarks cannot detect this failure by construction.

COMMON MISTAKE

Assuming a dateless model ID is an evergreen alias

Symptom. A team avoids `claude-opus-5` in production because they believe it will silently move to a newer model, and pins something they believe is more stable. Or the reverse: a team uses `claude-sonnet-4-5` expecting stability and observes behaviour changing under them.

Cause. The naming convention changed. Anthropic's documentation is explicit that for the 4.6 generation onward, the dateless ID *is* the pinned snapshot, while for earlier models the short form is a convenience alias that resolves to the latest dated snapshot.

Fix. For 4.6-generation and later models, the dateless ID is correct and pinned. For earlier models, use the full dated form.

Detection. Log the model ID returned in the API response on every call and alert on any change from what you configured.

COMMON MISTAKE

Comparing parameter counts across model families

Symptom. A selection decision is justified with "it is a 46B model, so it should beat our 13B model", and the deployed result is worse, slower or both.

Cause. Four incompatible things get called "parameter count": total for a dense model, total for an MoE model, active for an MoE model, and counts computed with or without embeddings. Mixtral's 46.7B total runs 12.9B per token. Separately, training budget dominates architecture: a 70B model trained on 15 trillion tokens beats an identical 70B trained on 300 billion by a margin no dimension change would produce.

Fix. Compare on measured task performance using the harness of Chapter 8. Use parameter counts for memory and cost arithmetic only.

Detection. Any decision document that cites a parameter count as evidence of quality without an accompanying measurement.

COMMON MISTAKE

Believing a pinned model ID freezes behaviour

Symptom. Outputs shift subtly on an unchanged model ID and unchanged prompt. The team spends days looking for a deployment that changed on their side.

Cause. Pinning fixes the weights. Anthropic's documentation states that serving infrastructure — router, safety classifiers, sampling logic — can change, and that such updates occasionally produce minor observable behavioural differences on an unchanged model ID.

Fix. Run the regression suite of Chapter 24 on a schedule, not only at upgrade time. Alert on distribution shift in output length, refusal rate and format-validation failures.

Detection. Continuous evaluation. There is no other way to see it.

COMMON MISTAKE

Quantising and shipping without task-specific evaluation

Symptom. A quantised deployment passes casual review, benchmarks look within noise, and a specific production capability — arithmetic, exact recall, JSON validity — degrades measurably.

Cause. Quantisation loss is unevenly distributed. Fluent conversational text is robust; long arithmetic chains, rare-fact recall and strict output formats are not. Aggregate benchmarks average the damage away.

Fix. Evaluate the quantised checkpoint against the full-precision one on your own task, with the sample sizes Chapter 8 requires — a twenty-example comparison establishes nothing.

Detection. Track schema-validation failure rates and numeric-accuracy rates before and after the precision change, as separate metrics from an aggregate score.

COMMON MISTAKE

Optimising attention because attention is the famous part

Symptom. Engineering effort goes into attention-related micro-optimisation while the feed-forward layers, which are 80% of the parameters, and the KV cache, which is the binding memory constraint, go unexamined.

Cause. Attention is what the papers, courses and diagrams emphasise, so it feels like where the model is.

Fix. Profile first. In a dense 70B model, 80% of the weights are feed-forward and the dominant serving constraint at long context is KV memory. Both are attention-adjacent rather than attention itself.

Detection. Compare the parameter and memory breakdown of section 9.3 against where your last month of optimisation effort went.

9.19 Performance Considerations

Decode is memory-bandwidth-bound, and parameter count sets the floor. Chapter 4's section 4.9 established that decoding is sequential and bandwidth-bound. Every generated token requires reading the active parameters from memory. A 70B model at bf16 must stream 131.4 GiB per token from accelerator memory; at fp8 it is 65.7 GiB. This is the mechanism by which quantisation improves latency as well as capacity, and it is why MoE models decode faster than their total size suggests — only the active experts are read.

Prefill is compute-bound, and context length sets the cost. The quadratic of section 4.10 applies here, so time-to-first-token grows faster than linearly with prompt length. Prompt caching addresses exactly this: it skips prefill for a cached prefix, reducing time-to-first-token and input cost, and does nothing for decode.

Continuous batching is the largest single throughput lever and it lives in the serving stack. Naive batching waits for the slowest sequence in a batch to finish. Continuous batching admits a new sequence the moment one completes. Paged KV allocation, which stores the cache in fixed-size blocks rather than contiguous per-sequence buffers, removes the fragmentation that otherwise wastes a large fraction of cache memory. Neither is a property of the model; both dominate the numbers this chapter computes. Treat the concurrency figures here as an upper bound that a good serving stack approaches and a bad one does not.

Layers are sequential and add latency that hardware cannot remove. An 80-layer model has 80 dependent steps per token. Tensor parallelism splits each layer across accelerators and adds a network round trip per layer; pipeline parallelism splits layers across accelerators and adds latency per stage. This is why crossing from one accelerator to two, as section 9.6 showed quantisation can avoid, is a latency decision and not only a memory one.

On the Claude API, none of this is yours — but all of it is priced in. You cannot batch, quantise or shard. What you can do is control token counts, structure prompts so that the stable prefix is cacheable, and choose a model tier. Chapter 11 covers the latency and throughput model in full; Chapter 30 covers batching and caching as API features rather than serving-stack features.

9.20 Security Considerations

Model weights are the most concentrated intellectual property in the system. A checkpoint is a single artefact worth the entire training run. If you self-host, the weights file needs the access controls, encryption at rest and audit trail you would apply to a production database dump — and copying it is a single `cp`. The commonest exposure is not an attacker but a convenience: weights on a shared filesystem so that several teams can experiment.

The KV cache contains user data in a recoverable form. It is derived from the tokens of the conversation, which include everything the user sent and everything your system placed in the prompt. Cache memory that is not zeroed between sequences is a cross-tenant leak, and cache offloaded to host memory or disk for capacity reasons is user data at rest in a place your data-handling review probably has not considered. Ask your serving stack what it does on sequence completion, and ask again about any paging tier.

Prompt caching is a shared-prefix mechanism, so scope it deliberately. The mechanism of section 4.9 means a cached prefix is reusable by anything that presents the same prefix. Never place one tenant's data in a prefix that another tenant's request could match. Chapter 30 covers the API's isolation guarantees; the architectural rule is simpler — cache what is *common* by design, never what is *shared* by accident.

Quantisation changes the safety surface, not only the quality surface. Refusal behaviour and jailbreak resistance are learned behaviours encoded in the weights, and aggressive quantisation perturbs them. A quantised model's safety evaluation is not inherited from the full-precision model's. Re-run it,

with the layered architecture of Chapter 8 in place either way, because model alignment was never the control you depended on.

Model IDs belong in your audit log. When you must reconstruct what a system did six months ago, the model ID is the difference between an explanation and a hypothesis. It is also, with a pinned ID, a reproducibility claim you can partially honour — subject to the serving-infrastructure caveat of section 9.10.

9.21 Real Production Examples

A document-intelligence platform discovers its context ceiling. A team serving contract analysis on self-hosted 70B weights advertised 128K-token analysis and provisioned for 60 concurrent users on two nodes. Real usage clustered near the maximum context, and effective concurrency was 22 rather than 60 — the arithmetic of section 9.8, arriving as a queueing incident. The fix was not more hardware. It was a two-stage architecture: retrieve the relevant clauses with the pipeline of Chapter 5, then send 20K tokens rather than 128K. Concurrency went up roughly six-fold, latency improved because prefill shrank, and quality improved because attention was no longer diluted across a hundred pages of boilerplate.

A migration blocked by a tokenizer, not a model. A team upgrading across the Claude 4.7 tokenizer boundary found their cost model wrong by roughly 30% and several prompts newly exceeding a length budget they had tuned against the previous tokenizer. Nothing about the model's quality was the problem. The lesson is section 9.11's first item: the tokenizer is part of the deployment, it changes between generations, and `count_tokens` is free.

An alias assumed to be a pin. A team using a pre-4.6 short-form model ID observed a behavioural shift and searched their own deployment history for a cause. The alias had resolved to a newer dated snapshot. The remediation was three lines of configuration; the cost was two days of investigation and a lost weekend. The generalisable control is the detection step from section 9.18: log the model ID returned on every response and alert on divergence from what you configured.

Quantisation that passed review and failed production. A team moved a self-hosted model to int4 to fit a single accelerator. Aggregate benchmark scores moved within noise. In production, JSON schema-validation failures rose from 0.4% to 3.1%, because strict format adherence is exactly the capability aggressive quantisation degrades first and exactly the capability an aggregate benchmark averages away. The corrective was to evaluate the specific capability, at the sample sizes of Chapter 8, rather than the aggregate.

9.22 Certification Notes

Concept in this chapter	CCAR-F domain	How it is examined
Model IDs, aliases, dateless pinning	D5 Context Management and Reliability	Scenario: a team observes behavioural drift and must identify the cause and remediation
Serving infrastructure can change under a pinned ID	D5	Distractor question: "pinning guarantees identical outputs"

Concept in this chapter	CCAR-F domain	How it is examined
Context length as a memory and cost decision	D5, D1 Agentic Architecture	Scenario: capacity planning, or choosing retrieval over a longer window
KV cache growth and prompt caching mechanics	D5	Why caching must be prefix-only and accelerates prefill only
Retrieval versus context-stuffing	D5, D1	The most common wrong answer in Domain 5, per Chapter 4's exam note
Model selection on measurement, not size	D1	Scenario: a smaller model is the correct answer on cost and latency
Tokenizer changes between generations	D4 Prompt Engineering and Structured Output	Cost estimation, and prompts near a length budget
Parameters as memory, not capability	D1	Distractor: "choose the largest model"
Total versus active parameters (MoE)	D1	Which governs memory, which governs compute
The model is a component, not the system	D1, D5	Recurs throughout; the framing behind most scenario answers

CERTIFICATION NOTE

What Domain 1 and Domain 5 actually reward here

The exam does not ask you to count parameters. It asks you to make decisions in which the anatomy is the hidden constraint.

Three patterns recur, and each has a distinctive shape:

"Choose a model for this workload." The correct answer is almost never the largest one. It is the smallest model that meets a *measured* requirement, chosen on cost and latency. Any option justified by size alone is a distractor. Any option that proposes measuring on your own task is a strong candidate.

"The system behaves differently than last month." Work the list: tokenizer change, model ID or alias movement, serving-infrastructure update, prompt or context change on your own side. The distractor is usually "the model was retrained" — models are not retrained under an existing ID.

"We need to support longer documents." The correct answer involves retrieval, chunking or compaction, not a larger context window. The reasoning is quadratic compute cost (Chapter 4), linear KV memory cost (this chapter), and degraded mid-context retrieval (Chapter 55). Being able to state all three distinguishes a pass from a good pass.

Do not memorise parameter counts for Claude models. They are not published, they are not examinable, and any source that offers one is speculating.

9.23 Practice Questions

1. A model has `d_model = 8192`, 80 layers, 64 query heads, 8 key/value heads and `head_dim = 128`. Which pair of matrices is smallest? A) `Wq` and `Wo` B) `Wk` and `Wv` C) the feed-forward matrices D) the embedding matrices

2. In a dense 70B model, approximately what share of parameters is in the feed-forward layers? A) 17% B) 40% C) 55% D) 80%

3. Doubling `d_model` while holding layer count fixed multiplies block parameters by approximately: A) 1× B) 2× C) 4× D) 8×

4. Mixtral 8x7B has 46.7B total and 12.9B active parameters. Which resource is governed by the *total*? A) FLOPs per token B) Weight memory C) Decode latency D) KV cache size

5. Moving weights from bf16 to fp8 halves weight memory. What is the usual *architectural* benefit? A) Higher quality B) Longer context window C) Fitting on fewer accelerators, removing an inter-device hop D) A smaller KV cache

6. Which of these is **not** counted in a model's parameter count? A) Attention projection matrices B) The KV cache C) Embedding matrices D) Normalisation scale vectors

7. KV cache size scales how with context length? A) Constant B) Linear C) Quadratic D) Logarithmic

8. A 70B GQA model at bf16 needs 39.1 GiB of KV cache per 128K-token sequence. Two 80 GiB accelerators hold the 131.4 GiB of weights. Approximately how many concurrent 128K sequences fit? A) 0 B) 4 C) 16 D) 64

9. Which change most increases concurrency for a memory-bound long-context deployment? A) Quantising weights to fp8 B) Quantising the KV cache to fp8 C) Adding a faster interconnect D) Increasing batch size

10. According to Anthropic's documentation, `claude-opus-5` is: A) An evergreen alias to the latest Opus B) A pinned snapshot C) A convenience pointer to a dated ID D) A deprecated identifier

11. According to Anthropic's documentation, `claude-sonnet-4-5` (pre-4.6 generation) is: A) A pinned snapshot B) An alias resolving to the most recent dated snapshot C) Invalid D) A Bedrock-only ID

12. A team pins a model ID and still observes minor behavioural changes. The most likely documented cause is: A) The weights were updated B) A serving-infrastructure update C) The tokenizer changed D) Their prompt changed

13. Which is **not** part of the served model as described in Figure 9.1? A) Tokenizer B) Sampler C) Optimiser state D) Safety classifiers

14. Passing a non-default `temperature` to Claude Opus 5 returns, according to the official documentation at the time of writing: A) A warning B) A 400 error C) Silent clamping D) Higher latency

15. Claude Haiku 4.5's context window, per the documentation, is: A) 200K tokens B) 500K tokens C) 1M tokens D) 2M tokens

16. Why is GQA described as what made long context physically possible? A) It reduces the number of layers B) It reduces KV cache size by the query-to-KV head ratio C) It removes the quadratic in attention D) It compresses the weights

17. A model's parameter count reliably predicts which of the following? A) Quality on your task B) Context window length C) Weight memory D) Tool-use reliability

18. The best evidence that an anatomy calculator is correct is: A) It produces round numbers B) It matches published totals for several real models exactly C) It agrees with a blog post D) It runs quickly

19. For a memory-bound long-context deployment, the highest-return lever is usually: A) Weight quantisation B) Reducing context length through retrieval C) A faster accelerator D) More layers

20. After a tokenizer change between model generations, the correct action is: A) Scale old counts by a fixed factor B) Re-measure with `count_tokens` C) Ignore it, since billing is unaffected D) Switch to character counts

9.24 Answers and Explanations

1 — B. With 8 key/value heads against 64 query heads, `Wk` and `Wv` are one-eighth the size of `Wq` and `Wo`. **A** is wrong because those two use the full query head count. **C** is wrong by a wide margin — feed-forward is roughly 80% of the model. **D** is wrong: embeddings are 2.1B here, larger than `Wk` and `Wv` combined across all layers.

2 — D. Measured at 79.90% for Llama 3.1 70B. **A** is the *attention* share (17.12%) — a deliberate swap, and the commonest wrong answer for anyone who remembers that attention is the important mechanism.

3 — C. Every block matrix has `d` on at least one side and usually both, so block parameters scale with d^2. **B** is the *depth* answer: doubling layers doubles parameters linearly.

4 — B. Every expert must be resident because you cannot predict which token routes where. **A** and **C** follow the active count, since only selected experts run. **D** is wrong: KV cache depends on layers, kv heads, head dimension and tokens — the feed-forward experts do not enter it at all.

5 — C. The gain is usually the topology change, not the bytes: one accelerator instead of two removes an inter-device hop from every forward pass. **A** is backwards — quality degrades slightly. **B** is unrelated; context length is a training and serving property. **D** is wrong: cache precision is an independent choice.

6 — B. The cache is allocated at serve time and is not learned, which is exactly the omission that caused the introduction's failure. **D** is a tempting distractor because norms are tiny — 1.3M out of 70.5B — but they are learned and they are counted.

7 — B. Linear: `2 × layers × kv_heads × head_dim × tokens × bytes`. **C** confuses cache *memory* with attention *compute*, which is genuinely quadratic (section 4.10). Both facts are true; they describe different resources, and telling them apart is the point of the question.

8 — A. Zero. After the weights and a 10% reserve, 12.6 GiB remains against a 39.1 GiB requirement. This is the chapter's opening incident, and the fact that the intuitive answer is "surely at least a few" is why it happens.

9 — B. The cache is a *per-sequence* cost, so halving it doubles concurrency; the measured figures are 11 → 13 for weights alone and 11 → 26 with the cache. **A** frees a fixed amount once. **C** and **D** do not change memory arithmetic — and **D** is what you are trying to increase, not a means of doing so.

10 — B. From the 4.6 generation onward, dateless IDs are pinned snapshots, not evergreen pointers. The documentation exists specifically to correct **A**, which is the intuitive reading of the name.

11 — B. Before 4.6, short forms are convenience aliases resolving to the most recent dated snapshot. Note that this is the opposite of question 10 — same-looking name, different generation, different guarantee.

12 — B. The documentation states that serving infrastructure can change and that such updates are the most likely cause of unexpected differences on a stable ID. **A** is explicitly contradicted: Anthropic does not update weights under an existing ID.

13 — C. Optimiser state exists during training and is discarded before serving. **B** is a good distractor: the sampler is genuinely part of the *deployment* while not being part of the *model*, which is the distinction section 9.7 draws.

14 — B. A 400 error, per the documentation at the time of writing. Verify on every upgrade; this is a generation-level behaviour, not a permanent API property.

15 — A. 200K tokens, against 1M for Fable 5, Opus 5 and Sonnet 5 — and note that this ordering is not a size ordering, which is the point of question 17.

16 — B. GQA cuts the cache by the query-to-KV head ratio: 8× for this configuration. Without it, a 1M-token sequence on a 70B model would need 2.4 TB. **C** is wrong — the quadratic in attention compute is unaffected by GQA.

17 — C. Weight memory is exactly `parameters × bytes_per_parameter`. **B** is independent of size: Haiku 4.5 at 200K and Opus 5 at 1M are not ordered by any published dimension. **A** and **D** are training outcomes, not architectural ones.

18 — B. Exact agreement with published totals for three independent architectures — dense small, dense large, sparse — is external validation. **A** is actively suspicious: real totals are not round.

19 — B. Retrieval attacks the per-sequence cost, which dominates at long context. Measured in the mini project: 0 → 16 concurrent from a 128K-to-32K reduction on unchanged hardware, against 0 → 2 for weight quantisation.

20 — B. Counts are model-specific; `count_tokens` is free and rate-limited independently of the Messages API. **A** is wrong because the ~30% figure is an average across content types and languages, not a constant. **C** is wrong: billing reflects the change.

9.25 Ten Practical Exercises

1. Transcribe three open-weight model configs into `Arch` objects and verify each total against its published figure.
2. Plot KV cache against context length for MHA, GQA and MQA on one chart, with weight memory as a horizontal line, and mark the three crossover points.
3. Extend the calculator to report **training** memory, including gradients and two Adam moments per parameter, and report the ratio to inference memory.
4. Add an activation-memory estimate for prefill as a function of batch size and prompt length, and re-derive concurrency with it included.
5. Compute the token count at which prompt caching pays for itself given a cache-write premium and a cache-read discount, then check it against Chapter 30's pricing.
6. Write a function that, given a target concurrency and context length, returns the cheapest valid combination of accelerator count and precision pair.

7. Take a production prompt, measure it with `count_tokens` against two model generations, and quantify the tokenizer difference on your own text.
8. Audit your codebase for model IDs. Classify each as pinned or aliased, and list the ones that are hard-coded rather than configured.
9. Build a small dashboard that logs the model ID from every API response and alerts on any change.
10. Write the one-page capacity note you would bring to a design review proposing a context-length increase, using the calculator's output.

9.26 Five Challenge Labs

1. **Sensitivity analysis.** For a fixed parameter budget of 30B, sweep width against depth and report parameter count, estimated decode latency and KV cache size for each configuration. Identify the shape of the trade-off surface and say which corner you would ship.
2. **The MoE decision.** Given a memory budget, a compute budget and a latency target, write a function that decides between a dense model and an MoE model of equivalent active size, and justify each branch with the arithmetic.
3. **Cache-aware admission control.** Implement a scheduler that admits requests only when the projected KV cache fits the remaining budget, using the calculator to project. Show it converting an OOM into a queue.
4. **Precision impact harness.** Using the evaluation methodology of Chapter 8, design an experiment that would detect a 2-percentage-point regression from quantisation on structured output. State the sample size and justify it with a confidence interval.
5. **Upgrade rehearsal.** Write a runbook for migrating a production system to a new model ID: what to measure before, what to compare, what the rollback trigger is, and how you would detect the serving-infrastructure drift of section 9.10 separately from a genuine model change.

9.27 Interview Questions

1. What does a model's parameter count tell you, and what does it not?
2. Why is the KV cache not included in a parameter count, and why does that matter?
3. Explain grouped-query attention to someone who has never seen a transformer, and say what it saves.
4. A 70B model at bf16 fits on two 80 GiB accelerators. How many 128K-token conversations can it serve concurrently, and how would you find out?
5. What is the difference between total and active parameters, and which governs cost?
6. Why is width scaling quadratic and depth scaling linear? Which would you choose and why?
7. What changes between model generations that you can actually observe and manage?
8. A pinned model ID's behaviour shifts. Walk me through your investigation.
9. When would you quantise a model, and what would you measure before shipping it?
10. Your product wants to raise the context limit from 32K to 128K. What do you tell them?

9.28 Summary

A parameter count is a floor area. It tells you what the deployment weighs, what it costs to hold and how many machines it needs, and it tells you nothing reliable about whether the model is any good at your task. That distinction — Chapter 3's capacity against capability, arriving here with numbers attached — is the first thing to carry out of this chapter, because the most common architectural error involving model size is treating it as a quality signal. Quality is a training outcome. Size is a memory fact.

The arithmetic behind the fact is small enough to hold in your head and precise enough to be checked. Embeddings scale as `vocab × d`; attention as `d²` with the key and value projections shrunk by the query-to-KV head ratio; feed-forward as `3 × d × ffn_hidden`, which in a large dense model is about 80% of everything. Width costs quadratically and depth costs linearly, but depth costs latency that no hardware removes, so real architectures sit in a narrow band around a hundred to one. Mixture-of-experts breaks the single number into two: total parameters, which you must hold in memory, and active parameters, which you pay for in compute — and the deployment cost follows the total, not the active count, which is the reverse of what the marketing implies.

The served model is seven components and the weights are one of them. The tokenizer determines your bill before anything runs and changes between generations. The sampler is not part of the model at all, which is why the same weights can be deterministic or creative, and why the Claude API's 400 on non-default sampling parameters is an interface decision rather than a model property. The serving stack — batching, paged allocation, scheduling — governs throughput more than the weights do. And the safety classifiers and routing layer sit around all of it. On the API you own the prompt, the context length, the model ID and the caching strategy; self-hosting means owning all seven, including the upgrade risk.

The component that is always forgotten is the KV cache, and it is the one that ends capacity plans. It is linear in context length and it is charged per sequence rather than shared, so at long context it dominates. On a 70B GQA model, a single 128K-token conversation costs 39.1 GiB and a 1M-token conversation costs 305 GiB — 2.3 times the entire model. Built with plain multi-head attention the same 1M-token sequence would need 2.4 terabytes, which is why grouped-query attention is not a micro-optimisation but the thing that made long context physically possible. The consequence for design is a clean rule: when memory-bound, attack per-sequence costs before shared costs. Our measurements put weight quantisation at 0 to 2 concurrent sequences and a context reduction from 128K to 32K at 0 to 16, on identical hardware. Retrieval is a capacity upgrade, not only a quality one.

Finally, the vocabulary that survives contact with an incident review. A model is an architecture and a recipe; a checkpoint is one frozen set of values; a deployment is that checkpoint plus everything around it; an alias is a name that may or may not move. Anthropic's documented scheme inverts the intuition: `claude-sonnet-4-5` moves and `claude-sonnet-5` does not, because dateless IDs from the 4.6 generation onward are pinned snapshots rather than pointers. Pinning fixes the weights and not the behaviour — the documentation is explicit that serving infrastructure can change and is the likeliest cause of drift on a stable ID. So pin explicitly, treat a model ID change as a code change, log the ID on every response, and keep the regression suite running on a schedule. You cannot manage what you are not told; manage what you are told, and measure the rest.

KEY TAKEAWAYS

1. **A parameter count is memory, not merit.** It predicts weight memory exactly and quality not at all.
2. **Feed-forward is ~80% of a large dense model**; attention is ~17%. Cost share and parameter share differ.
3. **Width scales quadratically, depth linearly** — but depth adds latency hardware cannot remove.
4. **For MoE, memory follows total parameters and compute follows active.** Deployment cost follows total.
5. **Precision is a multiplication with a topology consequence**: the win is usually fitting on fewer accelerators.
6. **A served model is seven components**; the weights are one. The sampler is not part of the model.
7. **The KV cache is linear in context and charged per sequence.** At 1M tokens it exceeds the model itself.
8. **GQA is what made long context possible** — 8× here, and 2.4 TB per sequence without it.
9. **When memory-bound, attack per-sequence costs first.** Retrieval beat weight quantisation eight-fold in our measurements.
10. **Dateless IDs from 4.6 onward are pins, not aliases** — and pinning fixes the weights, never the behaviour.

9.29 Glossary

Term	Definition
Active parameters	The parameters a single token flows through; equals the total in a dense model, far less in an MoE model.
Alias	A model name that resolves to a checkpoint; may move. Pre-4.6 Claude short-form IDs are aliases.
bf16	16-bit "brain float"; fp32's exponent range with fewer mantissa bits. The de facto serving standard.
Checkpoint	One specific set of frozen parameter values produced by a training run.
Concurrency ceiling	The number of simultaneous sequences a memory budget supports, after weights and overhead.
Deployment	A checkpoint plus its serving stack, sampler, classifiers and routing, in a particular region.
Expert	One of several parallel feed-forward networks in an MoE block.
fp8 / int4	8-bit float and 4-bit integer weight formats, 1 byte and 0.5 bytes per parameter.
GQA	Grouped-query attention: several query heads share one key/value pair, shrinking cache and weights.
KV cache	Per-sequence store of keys and values for every processed token; linear in context length, not a parameter.
Mixture of experts (MoE)	An architecture with several feed-forward experts per block, of which a router selects a few per token.

Term	Definition
Model ID	The string identifying a model in an API request. From the Claude 4.6 generation onward, a pinned snapshot.
Parameter	A single number learned in training and frozen at inference.
Pinned snapshot	A model ID whose underlying weights are fixed for the lifetime of the ID.
Quantisation	Converting weights, or a KV cache, to a lower-precision format to save memory.
Router	The small matrix in an MoE block that scores experts for each token.
Sampler	The parameterless code that selects a token from the model's output distribution.
SwiGLU	A gated feed-forward variant with three matrices — gate, up and down — rather than two.
Tied embeddings	Sharing one matrix between input embedding and output projection, saving `vocab × d` parameters.
Total parameters	Every parameter in the model, including experts that a given token does not use.

9.30 Further Reading

Primary sources. Anthropic's *Models overview* and *Model IDs and versioning* pages on `platform.claude.com` are the authoritative statements on the model lineup, context windows, maximum output, pinning semantics and the serving-infrastructure caveat — all of them re-verified for this chapter. The *Context windows* page covers what counts toward the window, overflow behaviour and the `model_context_window_exceeded` stop reason. For the architecture itself, Vaswani et al., *Attention Is All You Need* (2017) remains the source; Shazeer's *Fast Transformer Decoding* (2019) introduced multi-query attention and Ainslie et al.'s *GQA* (2023) the grouped variant. For open-weight configurations you can check arithmetic against, the Llama and Mixtral model cards publish full dimensions.

Where to go next in this book. Chapter 10 takes the anatomy and asks what it can and cannot do — capabilities and hard limits, and why some failures are structural rather than fixable. Chapter 11 turns the memory arithmetic here into latency, throughput and cost models. Chapter 12 explains, via scaling laws, why training budget dominates architecture, and how benchmarks should and should not be read. Chapter 13 covers reasoning models and test-time compute — the alternative to spending more parameters. Chapter 14 introduces the Claude model family concretely and Chapter 15 turns model choice into a procedure. The KV-cache arithmetic returns in Chapter 19 as rate limits, in Chapter 30 as prompt caching and cost controls, and in Chapters 52 to 55 as the context budget that governs every agentic system you will build.